

РЕШЕН ИЗПИТ С ПЪЛНИ ОБЯСНЕНИЯ

- Вариант 1

СЕКЦИЯ 0 — ПРОЧИТ И АНАЛИЗ НА ЗАДАЧАТА

Преди да напишем и един ред код, анализираме задачата методично. Тази стъпка е ключова, за да дефинираме правилните типове данни и файлови режими.

ИЗРАЗ ОТ УСЛОВИЕТО	КАКВО ОЗНАЧАВА ЗА НАС В С
"уникален номер... десетцифрено цяло число"	long long id; Стандартният int събира до ~2.14 милиарда (10 цифри, но не всички). За сигурност използваме long long с формат %lld.
"низ с дължина 128 символа"	char name[129]; Винаги добавяме +1 за нулевия терминатор '\0'. Никога не използваме char* в структура, която ще се записва във файл!
"масив с цели числа - номерата на курсовете"	long long *course_ids; Тъй като броят на записаните курсове се променя, това трябва да е динамичен масив (показалец), заделен с malloc.
"генерира текстов файл course_info.txt"	FILE *f = fopen("course_info.txt", "w"); Използваме режим "w" (write), защото генерираме файла наново по зададени критерии.
"добавя информацията в двоичен файл course_student.bin"	FILE *f = fopen("course_student.bin", "ab"); Използваме режим "ab" (append binary). "a" за добавяне в края, "b" задължително за двоичен файл.

СЕКЦИЯ 1 — ПЪЛЕН КОД

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```
// --- СТРУКТУРИ ---
```

```
typedef struct {  
    long long id;  
    char name[129];  
    char teacher[65];  
    char date[11]; // mm.dd.yyyy + '\0'  
    int students_count;  
} Course;
```

```
typedef struct {  
    char name[65];  
    long long fac_num;  
    int courses_count;  
    long long *course_ids; // Динамичен масив за номерата на курсовете  
} Student;
```

```
// --- ПОМОЩНА ФУНКЦИЯ ЗА ДАТИ ---
```

```
// Връща: > 0 ако date1 > date2; < 0 ако date1 < date2; 0 ако са равни
```

```
int compareDates(const char* date1, const char* date2) {
```

```
    int d1, m1, y1, d2, m2, y2;  
    sscanf(date1, "%d.%d.%d", &d1, &m1, &y1);  
    sscanf(date2, "%d.%d.%d", &d2, &m2, &y2);
```

```
    if (y1 != y2) return y1 - y2;  
    if (m1 != m2) return m1 - m2;  
    return d1 - d2;  
}
```

```
// --- ЗАДАЧА 2: Генериране на текстов файл ---
```

```
int generateCourseInfo(Course *courses, int count, const char *teacher, const char  
*start_date, const char *end_date) {
```

```
    FILE *f = fopen("course_info.txt", "w");  
    if (f == NULL) return -1;
```

```
    int match_count = 0;  
    for (int i = 0; i < count; i++) {  
        // Проверка за преподавател  
        if (strcmp(courses[i].teacher, teacher) == 0) {  
            // Проверка дали датата е в периода (start_date <= date <= end_date)
```

```

        if (compareDates(courses[i].date, start_date) >= 0 &&
compareDates(courses[i].date, end_date) <= 0) {
            fprintf(f, "%lld:%s:%s-%d\n", courses[i].id, courses[i].name, courses[i].date,
courses[i].students_count);
            match_count++;
        }
    }
}
fclose(f);
return match_count;
}

```

// --- ЗАДАЧА 3: Записване на студент (Двоичен файл) ---

```

int enrollStudent(Course *courses, int c_count, Student *students, int s_count, long long
fac_num, long long course_id) {

```

```

    // 1. Намиране на студента и курса

```

```

    Student *s = NULL;

```

```

    Course *c = NULL;

```

```

    for(int i=0; i<s_count; i++) {

```

```

        if(students[i].fac_num == fac_num) { s = &students[i]; break; }

```

```

    }

```

```

    for(int i=0; i<c_count; i++) {

```

```

        if(courses[i].id == course_id) { c = &courses[i]; break; }

```

```

    }

```

```

    if(!s || !c) return -1; // Системна грешка (не са намерени)

```

```

    // 2. Бизнес логика проверки

```

```

    if (c->students_count >= 30) return 0; // Няма свободно място

```

```

    if (s->courses_count >= 10) return 0; // Максимум 10 курса

```

```

    for(int i=0; i<s->courses_count; i++) {

```

```

        if(s->course_ids[i] == course_id) return 0; // Вече е записан

```

```

    }

```

```

    // 3. Запис в двоичен файл

```

```

    FILE *f = fopen("course_student.bin", "ab");

```

```

    if (!f) return -1;

```

```

    fwrite(&course_id, sizeof(long long), 1, f);
    fwrite(&fac_num, sizeof(long long), 1, f);
    fclose(f);

    // 4. Актуализиране на характеристиките в паметта
    c->students_count++;

    // Разширяване на динамичния масив с курсове на студента
    s->course_ids = realloc(s->course_ids, (s->courses_count + 1) * sizeof(long long));
    if(!s->course_ids) return -1; // Системна грешка при паметта

    s->course_ids[s->courses_count] = course_id;
    s->courses_count++;

    return 1;
}

// --- ЗАДАЧА 4: Изтриване по дата ---
// Връщаме новия размер на масива чрез return (или може с двоен указател)
int deleteOldCourses(Course **courses, int count, const char *target_date) {
    int new_count = 0;
    int deleted = 0;

    // In-place филтриране: премества валидните елементи напред
    for (int i = 0; i < count; i++) {
        if (compareDates((*courses)[i].date, target_date) >= 0) { // Ако НЕ Е преди
подадената
            (*courses)[new_count] = (*courses)[i];
            new_count++;
        } else {
            deleted++;
        }
    }

    if (deleted == 0) {
        printf("Няма открити курсове за изтриване.\n");
        return count; // Връщаме стария размер
    }
}

```

```

// Преоразмеряваме масива до новия, по-малък размер
if (new_count > 0) {
    Course *temp = realloc(*courses, new_count * sizeof(Course));
    if (temp) *courses = temp;
} else {
    free(*courses);
    *courses = NULL;
}

return new_count;
}

int main() {
    // Тук би се реализирала логиката по Задача 1 (въвеждане от потребител)
    return 0;
}

```

СЕКЦИЯ 2 — ОБЯСНЕНИЯ СЪПКА ПО СЪПКА

Анализ на Работата с Дати (Задача 2 и 4)

Когато датата е във формат низ (string) "dd.mm.yyyy", стандартната функция `strcmp()` **няма да работи правилно** за хронологично сравнение. Например `strcmp("01.12.2023", "02.01.2023")` ще каже, че първата е по-малка (заради '01' срещу '02'), въпреки че декември е след януари.

Решението: Пишем помощна функция `compareDates`. Използваме `sscanf`, за да "извлечем" числата за ден, месец и година, след което ги сравняваме логично: първо година, после месец, накрая ден.

Обяснение на Задача 3 (Свързване на структури)

Това е класическа M:N (много-към-много) връзка. Извършваме стъпките методично:

1. **Търсене:** Обхождаме масивите със студенти и курсове, за да намерим конкретните обекти чрез техните ID-та. Използваме указатели `Student *s` и `Course *c`. Защо указатели? Защото искаме да променим *оригиналните* данни в масива, а не техни копия!
2. **Валидация:** Проверяваме бизнес ограниченията (30 места в курс, 10 курса на студент, дублиране).
3. **Двоичен запис:** `fopen("course_student.bin", "ab")`. Пишем променливите директно с `fwrite`. Забележете: подаваме `&course_id` (адреса на променливата).
4. **Динамична памет в структура:** Тъй като студентът получава нов курс, трябва да

разширим неговия личен масив `course_ids`. Използваме `realloc`, аналогично на купуването на по-голям рафт за книги, когато старият се препълни.

СЕКЦИЯ 3 — ТОП 3 ГРЕШКИ В ТОЗИ ВАРИАНТ

- **Грешка 1: Изтичане на памет (Memory Leak) в Задача 1.** Когато създавате структурата `Student`, нейният масив `course_ids` първоначално трябва да се инициализира с `NULL`. Ако остане с произволна стойност, `realloc` в Задача 3 ще предизвика краш на програмата.
- **Грешка 2: Изтриване от масив (Задача 4).** Най-честата грешка е изтриването на елемент и оставянето на "дупка" в паметта или прескачането на индекс. Правилният подход е *"In-place filtering"* — имаме два индекса (за четене и за запис). Копираме валидните елементи един след друг и накрая режем масива с `realloc`.
- **Грешка 3: Сравняване на ID-та с `strcmp`.** Понеже условието казва "десетцифрено число", ние използваме `long long`. Следователно сравнението е директно: `if (c->id == course_id)`. Не се ползва `strcmp` за числови типове!